100
Pass

Poola Mohan

Other

PDF generated at: 3 Mar 2026 08:45:55 UTC

View this report on HackerRank

Score

100% • 225.0 / 225

scored in eClerx - FO Java, Unix & Python Assessment in 40 min 22 sec on 3 Mar 2026 13:33:33 IST

Candidate Information

Email	poolamohan10@gmail.com
Test	eClerx - FO Java, Unix & Python Assessment
Candidate Packet	View
Taken on	3 Mar 2026 13:33:33 IST
Time taken	40 min 22 sec/ 90 min
Work Experience	4 years
Invited by	Priyanshi
Invited on	2 Mar 2026 17:48:35 IST

Skill Distribution

No.	Skill	Score
1	Bash Intermediate	100%

2	Java Intermediate	100%
3	Python Intermediate	100%

Tags Distribution

Bash	100%	Medium	100%
Shell Scripting	100%	Arrays	100%
Strings	100%	Java	100%
Multithreading	100%	Interfaces	100%
OOP	100%	Context Manager	100%
Python	100%	Classes	100%

Questions

Linux/Unix • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Bash: Array Sum Coding	Bash (Intermediate)	75/75	-	16 min 51 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
2	Java: Safe Cracking Coding	Java (Intermediate)	75/75	-	11 min 33 sec	✓

Python (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
3	DB Context Coding	Python (Intermediate)	75/75	-	11 min 37 sec	✓

Candidate Feedback

Good

1. Bash: Array Sum

Correct

Coding Bash Medium Shell Scripting Arrays Strings

Question description

Given an array of strings, add all the numeric values within the array and print the sum to stdout. If there are no numeric values, print 0.

Note: The sum will not exceed 7,000,000.

Example:
my_array = ["First", "2", "3rd", "4"]

The *array_sum* function should return 6 because 2 + 4 = 6.

▼ INPUT FORMAT FOR CUSTOM TESTING

The input contains the newline-separated elements of the given array.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
Abc
20
Efg231
10
cDe
```

Sample Output

```
30
```

Explanation

The numerical elements are 20 and 10 that sum to 30.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
SingleWord
```

Sample Output

```
0
```

Explanation

No numeric elements are present in the array.

Candidate's Solution

Language used: **BASH**

```
#!/usr/bin/env bash
readarray -t my_array </dev/stdin

# Input array is read and stored in to my_array variable.
# You can view the code by pressing > button above.

function array_sum() {
```

```
  typeset -a data=("$@")
  # Write your code here
  sum=0
  for item in "${data[@]}";do
     if [[ "$item" =~ ^[0-9]+$ && -n "$item" ]];then
        sum=$((sum+=item))
     fi
  done
  echo "$sum"

}

array_sum "${my_array[@]}"
exit 0

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0029 sec	1.63 KB
TestCase 1	Easy	Sample	Success	1	0.0066 sec	1.63 KB
TestCase 2	Easy	Hidden	Success	1	0.0044 sec	1.88 KB
TestCase 3	Easy	Hidden	Success	6	0.0041 sec	1.88 KB
TestCase 4	Easy	Hidden	Success	6	0.004 sec	1.88 KB
TestCase 5	Easy	Hidden	Success	6	0.0036 sec	1.88 KB
TestCase 6	Easy	Hidden	Success	6	0.0251 sec	2.5 KB
TestCase 7	Easy	Hidden	Success	6	0.0261 sec	2.5 KB

TestCase 8	Easy	Hidden	Success	6	0.0297 sec	2.5 KB
TestCase 9	Easy	Hidden	Success	6	0.0276 sec	2.63 KB
TestCase 10	Easy	Hidden	Success	6	0.0289 sec	2.63 KB
TestCase 11	Easy	Hidden	Success	6	0.0272 sec	2.5 KB
TestCase 12	Easy	Hidden	Success	6	0.0272 sec	2.5 KB
TestCase 13	Easy	Hidden	Success	6	0.026 sec	2.5 KB
TestCase 14	Easy	Hidden	Success	6	0.0274 sec	2.5 KB

No comments.

2. Java: Safe Cracking

Correct

Coding Medium Java Multithreading Interfaces OOP

Question description

Implement a *SafeCracking* class that executes a sequence of operations to rob a bank vault. The class should coordinate with an *IActions* interface to perform the robbery in the correct order.

Your implementation must follow a specific sequence of operations:

1. First, unlock the vault door.
2. Then, simultaneously determine the safe deposit box number and hack its PIN.
3. Finally, open the safe deposit box using the obtained information.

Method to Implement

```
void rob(IActions actions)
```

This method should coordinate with the provided *IActions* object to execute the following steps in order:

1. Call *unlockTheDoor()* to get the door passcode string.
2. Concurrently call both:
   - *hackSecretPin(passcodeOfDoor)* to get the safe deposit box PIN
   - *figureOutSafeDepositBoxNumber(passcodeOfDoor)* to get the safe deposit box number
3. Call *openSafeDepositLock(safeDepositBoxNumber, safetyDepositBoxPin)* to complete the robbery.

Note: The concurrent operations in step 2 are critical for the time-sensitive nature of the robbery scenario.

Example

The system will validate your implementation by checking if the operations are called in the correct order with the proper parameters. A successful implementation will output:

```
true
```

This indicates that all operations were performed correctly, and the robbery was successful. If the operations are performed in the wrong order or with incorrect parameters, the output will be *false*.

Constraints

- $1 \leq$ lengths of *passcodeOfDoor*, *safeDespositBoxNumber, safetyDepositBoxPin* ≤ 10

▼ INPUT FORMAT FOR CUSTOM TESTING

There is no input passed from stdin.

▼ SAMPLE CASE 0

Sample Output

```
true
```

Explanation

- stdout shows *true* if it is a successful robbery otherwise, it shows *false.*

Interviewer guidelines

Java 8 and Java 15 Solution

```
class SafeCracking {
   public void rob(IActions actions) throws InterruptedException, ExecutionException{
      ExecutorService threadPool = Executors.newFixedThreadPool(2);
      String passcodeOfDoor = actions.unlockTheDoor();
      Future<String> hackSecretPinTask = threadPool.submit(() -> actions.hackSecretPin(passcodeOf
Door));
      Future<String> figureOutSafeDepositBoxNumberTask = threadPool.submit(() -> actions.figureO
utSafeDepositBoxNumber(passcodeOfDoor));

      String safeDespositBoxNumber = figureOutSafeDepositBoxNumberTask.get();
      String safeDepositBoxPin = hackSecretPinTask.get();

      actions.openSafeDepositLock(safeDespositBoxNumber, safeDepositBoxPin);
      threadPool.shutdown();
   }
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
```

```
    public boolean getRobberyVerdict();
}


class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{
        String passcodeOfDoor=actions.unlockTheDoor();
        ExecutorService executor=Executors.newFixedThreadPool(2);
        Future<String> pinFuture=executor.submit(()->{
            try{
                return actions.hackSecretPin(passcodeOfDoor);
            }catch(Exception e){
                return null;
            }

        });
         Future<String> boxFuture=executor.submit(()->{
            try{
                return actions.figureOutSafeDepositBoxNumber(passcodeOfDoor);
            }catch(Exception e){
                return null;
            }

        });
        String safetyDepositBoxPin=pinFuture.get();
        String safetyDepositBoxNumber=boxFuture.get();
        executor.shutdown();

        actions.openSafeDepositLock(safetyDepositBoxNumber, safetyDepositBoxPin);
    }
}
public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

```

```java
    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

```

```
    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	2.0532 sec	37.8 KB
Testcase 1	Hard	Hidden	Success	74	2.0553 sec	38.2 KB

No comments.

3. DB Context

Correct

Coding Medium Context Manager Python Classes OOP

Question description

Implement a class called *database_context_manager* that acts as a context manager for database connections.

The context manager should:

- Create a database connection when entering the context by calling the *_init_db()* method (already defined in the *Database* class)

- Close the connection when exiting the context by calling the *_close_db()* method (also defined in the *Database* class)

Given a string *username* consisting of lowercase English letters, define and complete the *database_context_manager class*.

For a successful simulation:

- The input string username must be passed to *_init_db()* as an argument.
- The connection string returned from *_init_db()* must be passed to *_close_db()* as an argument to close the connection.

Example

Given input *username* = "Sam", if the connection string returned by *_init_db()* is "abc":

- "Sam" must be passed as an argument to *_init_db()*
- "abc" must be passed to *_close_db()*

Function Description

Complete the class *database_context_manager* in the editor with the following parameter(s):

string username: the string to pass to *_init_db()*

Constraints

- $1 \leq |\ username\ | \leq 10^5$
- *username* consists of lowercase English letters.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line and the only line of the test case contains a string, *username*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
Robin
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Robin"*, the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.
- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
Sam
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Sam"*, the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.
- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

Interviewer guidelines

▼ SOLUTION

```
class database_context_manager():

    def __init__(self, username):
        self.username = username
        self.connectionString = ""
        self.obj = Database()
```

```
def __enter__(self):
    self.connectionString = self.obj._init_db(self.username)

def __exit__(self, exc_type, exc_value, exc_traceback):
    self.obj._close_db(self.connectionString)
```

Candidate's Solution

Language used: **Python 3**

```python
#!/bin/python

import math
import os
import random
import re
import sys

fptr = open(os.environ['OUTPUT_PATH'], 'w')

class Database:

    totalCount: int = 0
    count1: int = 0
    count2: int = 0
    inputString: int = ""
    __connString: int = ""

    def _init_db(self, username: str) -> str:

        Database.totalCount += 1
        Database.__connString = ""

        if username == Database.inputString:

            Database.count1 += 1

            le: int = random.randint(1, 100000)
            for x in range(le):
                nxt: int = random.randint(1, 100000)
                nxt %= 26
                Database.__connString += chr(97 + nxt)

            fptr.write('Username matched, database connection initialized\n')

```

```python
        return Database.__connString

    def _close_db(self, connectionString: str) -> None:

        Database.totalCount += 1
        if Database.__connString == connectionString:
            Database.count2 += 1
            fptr.write('Connection string matched, database connection closed\n')


#
# Complete the class 'database_context_manager' below.
#
# The class accepts STRING username as parameter.
#

class database_context_manager:

    def __init__(self, username):
        self.username = username
        self.db=Database()
        self.conn_string=None
    def __enter__(self):
        self.conn_string=self.db._init_db(self.username)
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.db._close_db(self.conn_string)




    # write your code here

if __name__ == '__main__':

    username: str = input()
    Database.inputString = username

    with database_context_manager(username) as dbSimulator:
        fptr.write('Database running...\n')

    tc: int = Database.totalCount
    cnt1: int = Database.count1
    cnt2: int = Database.count2
```

```python

    if cnt1 != 1 or cnt2 != 1 or tc != 2:
        fptr.write('Functions not correctly called\n')
    fptr.close()

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0865 sec	10.9 KB
TestCase 1	Easy	Sample	Success	1	0.2258 sec	11 KB
TestCase 2	Easy	Sample	Success	1	0.0614 sec	10.8 KB
TestCase 3	Easy	Hidden	Success	5	0.0819 sec	10.9 KB
TestCase 4	Easy	Hidden	Success	5	0.0939 sec	10.8 KB
TestCase 5	Easy	Hidden	Success	6	0.0957 sec	10.9 KB
TestCase 6	Easy	Hidden	Success	6	0.1764 sec	11 KB
TestCase 7	Easy	Hidden	Success	10	0.0298 sec	10.9 KB
TestCase 8	Easy	Hidden	Success	10	0.0544 sec	10.8 KB
TestCase 9	Easy	Hidden	Success	10	0.026 sec	10.9 KB
TestCase 10	Easy	Hidden	Success	10	0.0871 sec	10.9 KB

TestCase 11	Easy	Hidden	Success	10	0.1588 sec	11.1 KB

No comments.